- I created a light-weight SimpleUNet model architecture that works with the original researcher's model channel configurations. The encoder follows the doubling model the original researcher uses, but I simplified the setup for model efficiency, so I started from 64 then went up to 128 for the encoder, and then I went back down to 64 for the decoder. I specifically used a self-contained mode, and ddim_sample_loop function inspired by the Gaussian-Diffusion architecture of the Guided-Diffusion library. I used a self-contained version so as to maintain the fully self contained, lightweight structure of my inference script. Using a SimpleUNet was important for this study because it enabled us to have full control over all my variables. Since I was using pre-trained weights that had originally used the Guided-Diffussion library architecture, trying to work with the heavy guided-diffusion parameters would invalidate the study because I would not be able to get the proper parameters for the entire model architecture, so I would be trying to infer the necessary configurations for it, leading to potentially very biased results.

- To be able to accurately run predictions using the datasets, I followed the original researcher's method to crop and fed the SimpleUNet was fed the CT dimensions of the original researcher which is 256 width, 256 height, and a 3-slice channel. I did this for both test cases with SimpleITK by slicing the volumes to (z-1, z, z+1). I then decided to build back the model to a slice of 1 for 3D connectivity, effectively combining the prediction files with the other CT volumes to utilize the full abilities of the GATSegDiff's graph connectivity. This enables me to assess the model's ability to understand how the model interacts in the full 3D frame. While the original researcher reverts the volume back to 512*512, I chose to keep the cropped slice to 256, 256. I did this to best observe the predictions within the base pipeline.

- I implemented certain helper functions to better align the inference script with the methodology of the original researcher. This specific helper function mostly applies to the test images to help prime it for the model's architecture. I specifically implemented the original researcher's CT intensity cropping range of [0, 400] HU, which was used for both models as well. I also implemented the post processing function implemented by the original researcher to filter out noisy vessels for a clean output. To preserve the physical metadata when running the inference, I also added a resampling function to maintain real-world dimensions.

- The biggest difference in the inference methods I used came down to how the transposition of the volumes were handled. For my 3Dircadb1-originated CT volumes, I decided to automate the positioning of the segments so as to conform to the model's requirements of the order, which is (Slices, H, W), and then I added a function to turn it back to the original nIfTI order. decided to automate most of the metadata reading so as to properly align the images back to their original volume. The special functions were added because of the fundamental distinction between the 3Dircadb and the Task08 datasets, which is that the 3Dircadb is in DICOM format while Task08 is in nIfTI format.

For the 3Dircadb1 dataset CT volumes I needed to account for the structural metadata change that the volumes have overcome so as to not corrupt the data.

- For both ensemble scripts I decided to add a function to map the ensemble prediction back to the different original, unprocessed datasets. This step was important for evaluate my model ensemble predictions in accordance with the proper spacing dimensions provided by that variable dataset. I specifically decided to map my ensemble predictions back to the desired variable dataset because the purpose of those datasets are to test GATSegDiff' robustness and accuracy, and that can only be truly done if the desired unfiltered dataset wasn't there to give context. Also, because of the fact I were treating the GATSegDiff model as the standard, the model and the dataset it was ran on gets priority over nnUNet and the dataset nnUNet ran on. The reason I ended up having to use different ensemble scripts was because of the differences in the metadata. For the nnUNet predictions and GATSegDiff predictions on the 3Dircadb1 CT volumes I ended up having to create a dictionary label to be able to combine my volumes because the metadata and the numerical sequencing were very different from each other. The original plan was to map the segmentations all the way back to the 3Dircadb1 DICOM dataset, but then I realized that in doing that my data was bound to zero out and if I wanted to turn the ensemble prediction into binary masks their wouldn't be any label nodes since none of my voxels would show, so instead I had my prediction files map back to my 3D Nifti volumes inspired by the original 3Dircadb1 DICOM files. For the nnUNet predictions and GATSegDiff prediction on the Task08 Hepatic Vessel volumes, I decided to map back my volumes to the Task08 Hepatic Vessel dataset. I ended up having to edit my ensemble file as well for the Task08 Hepatic Vessel because the original script wasn't having it align to align the predictions, so the changes made were so that my data wouldn't zero out and that all necessary voxels were accounted for.

- I used a 0.5 baseline threshold for generating voxels within my binary label map for 3Dircab1 and for Task08 to eliminate noisy vessels that don't actually show the agreements of both models and ensure that my binary mass were still finding voxels for my data. I realized upon viewing my files originally that there was a lot of noise to my data for Task08 Hepatic Vessel, so upon inspection I realised it was because the Binary Mask threshold was way too low, specifically a value of 0.2. Knowing that keeping that threshold would cause practically every little segment to show and create a very unreliable result, I changed this threshold to (0.5) to make sure most of the voxels agreed with each other. This was done for both models. Then, specifically for my GATSegDiff predictions on MSD Hepatic Vessel, I saw that I had originally failed to take into account that Task08 not only segments the liver vessels, but it also segments tumors and other things as well, so my data was the result of capturing everything, leaving the vessels to be merged with the other segmented areas, so in the creation of my new ensemble binary mask, I made sure to only extract data that related to vessels to get more accurate results using a component-based noise cleaning function to erase the tumors and segmented background rather than a base noise cleaning function used the GATSegDiff's predictions on 3Dircadb1.

- I didn't use Dice-Based Model Ensembling because I am testing how the model's strengths are used (or not used) in each case, so I am basing my results on two different parameters, one based on a topological assessment of numerical metrics such as nodes, edges, loops, islands, millimeter length and topological stability, and a graph-based analysis in which I map out the accuracy of the voxel placements of the prediction nodes in accordance to the 3D volumetric graph with is the ground truth segmentation. This provides much more sound focus on the continuity of the fine hepatic vessel trees throughout the network of vessels which is important for volumetric extension and vessel connectivity.
-

- For analysis I are doing a qualitative assessment looking side by side at different images and observing the extension of the graph and the volumetric connectivity, which I define as the model's overall ability to maintain the interconnected network of the vessels preserving structural integrity. I also do a similar qualitative assessment on the volume's extensivity, which I define as the model's ability to map the fine vessels and the different bifurcations. The graph I originally had used were the label maps of 3Dircadb Vessel+, and I changed this because I realized that it would produce biases that would practically invalidate the entire study, given that the dataset is an extension of the base 3Dircadb1 dataset upon which my model ensemble production was built off of. After looking for a quality dataset to test off of, I decided it would do use good to create a graph using the Liver CT Deformable Registration Validation Dataset, which would contain no biases as it's a dataset the model ensemble has never encountered before, providing a good baseline for vessel segmentation performance. It is also highly appropriate because it would help analyze both the volumetric extensivity and connectivity of the ensemble. I did this by extracting the paired case and landmark numbers, keeping them paired. I used the anatomical landmarks to create the graph, but I use the case files to properly map each volume in accordance to how it is mapped against the volume. I strictly created edges within each anatomical landmark (my nodes) so as to keep spatial awareness within each case and produce a graph with the proper spatial dimensions.

- So upon further thought, I realized that using the Liver-DirQA dataset would sort of invalidate my entire study. This is not because the dataset itself is inappropriate for the study, but rather because my prediction utilized completely different ground truth segmentations. If I kept the original structure of my analysis, I would be in a situation where I would basically be comparing apples to oranges. They would never align in any way because the spatial coordinates are entirely different thanks to the likely use of different patients and different aims in the creation of these datasets. So I decided to use the current structure so I could get an honest review of the model's performance and not introduce unnecessary bias to my findings.

- The ensemble using nnUNet probabilities and GATSegDiff/MSD Hepatic Vessel probabilities struggled with noise handling. All the vessels are clumped to one slider and then there is a tiny fragment on the other side, leaving a large gap between the two segments. The vessel connectivity also proved to be an issue, as to the outskirts of vessels, the fine vessel trees, they are no longer connected to the rest of the vessel trunks. The hepatic artery and the hepatic vein are not distinct from each other, which makes it diffuser to pinpoint which one is which. The model also struggles with missing fine vessel trees, not just the really big one it missed. The hepatic artery is missing several extensive branches, and the portal vein is also missing key vessels as well. The only vessels that held any definition were the vessels that were big and recognizable. These findings show that the two main things this model struggled with was with volumetric extension and vessel definition.

- The ensemble using nnUNet probabilities and GATSegDiff/3Dircadb1 showed great work in vessel connectivity. Unlike the first model the vessels here are mostly linked together, with the fine vessel trees look much more connected in this model ensemble and have fewer false positives than did the first model ensemble. Likewise, from the side the volumetric connectivity looks to be very strong, given that the vessels maintain the even spread across the vasculature and the extension towards the branches. However if one looks more closely they'd see one big issue, an issue that manifests it self a little differently that in the first model. And that is that while to old metric connectivity looks good from one angle, at a different angle there is a gap that can be seen between two distance regions of vessels. This gap runs right through the middle between them. While in the first model everything was clumped up in a bunch, here it looks like the vessel is a packed in a way that forms a straight line path. So here it looks like problem is not volumetric extension, but volumetric connectivity.

- For my first set of two different tests I ran on both ensembles I created graph out of a dataset designed to to act as the ground truth for out ensemble. Some things I read for were Strict Hits, Proximity Halo points, and Distant Outliers. I define Strict Hits as predictions from the ensemble falling within a range of 3 voxels from the reference point. Proximity Halo Points are the predictions lying further than 3 voxels away but no more than 9 voxels away from the reference voxel point. Distance Outliers are termed for my study as predictions lying further than 9 voxels away, and they are generally viewed as outliers and noise because it is so far from the voxel point that it can no longer be seen as true indicators of a models ability to accurately predict the ground truth and are more less viewed as distracting.

- We realized that this setup would hurt my analysis of the data because the dataset I used to act as my ground truth would was never utilized in creating my prediction files, causing a severe mismatch that would artificially punish the models for lack of alignment and would hurt the credibility of my study. So I decided that I would try a direct Topological Stability overlap and graph the overlap between my ground truth

segmentation and my prediction. I ended up getting more data and more understanding of how my model did in predictions.

- I ended up having to scratch this out because I noticed that my ground truth segmentation nodes were looking more like a liver than the actual blood vessels themselves, and I realized that was because the ground truth dataset just contained the patient volumes rather than the segmentation labels. This goes back to a known problem particularly with MSD that only gives the patient volumes for their testing images and not the labels for them. So I ended up having to pivot towards unsupervised TDA, specifically using persistent homology to get past the data hurdle by analyzing the shape of my using the birth and death of data point and loops. I measured mean tortuosity, total vessel length, bifurcation count, average branch length, fragmentations (my h0 count), and false loops (my h1 count).

- So I like this idea, but I ran into an interesting problem. I had data, but I had no clue what this was supposed to mean for my model. To resolve this I first tried to go find the original patient scans and see what their original diagnoses were so I could use my descriptions that I scraped from my reading of the tumor cases noted in these datasets to generate a skeleton of the resulting vascular network and compare the two results for each patient record. The diagnoses were nowhere to be found in the metadata, only that 3Dircadb had 5 healthy patients. So I tried to use my TDA to deduce the disease profile using again my readings for the exact same methodological purpose. I tried to do each model separately, but I got vastly different results for the disease cases, which was a major issue for obvious reasons concerning scientific rigor. So I tried to do this for the ensembles for standardization but then I got almost all of one or two diseases. So I scratched that whole thing and decided to see if there were biophysical markers that don't change tumor or not, and that's how I found Murray's law. I plugged in the values and realized I kept getting different values that often deviated from Murray's deviation of 1.000 and Area Expansion of 1.26. That's where the biophysical markers of Valid Anatomy (the 1.000/1.26 baseline for Murray's deviation and Area Expansion respectively), Plausible tumor (0.45-1.000/1.26-2.00), and Impossible Hallucination (<0.45/>2.00) come into play. This gave me a way to actually benchmark my scores and see how the model is actually holding up.

- I utilized nnUNet's Generic UNet architecture backbones to track how the model dynamically handles the different tensor dimension using tensor land handling. I had originally wanted to create PyTorch simulations to model this but I realized that it may not be accurate to how nnUNet's Generic UNet actually works because a whole bunch of parameters would be missing, and I could be potentially making up certain parameters that for all I know nnUNet doesn't actually use for it's self-configuration.

- At first it was difficult to interpret the results because I didn't know how to connect my findings back to how the vascular dimension could be warped in the process of configuration. I then thought back to the linear algebra lesson I watched on elimination

matrices and reduced town echelon form. I remembered how the goal was always to simplify the data structure as much as possible and highlight the important data points. Likewise, I also thought about the TNB frames and arc length parameterization lessons as to how vector space curves were modeled. It hit me that what could potentially be happening is that the model could be highlighting the main vessel trunk that are typically centered in the middle of a vector space due to bifurcation patterns at the expense of the peripheral fine vessel tree dimensions. And because of this, the vector parameters that typically arise from these calculations may accurately model the middle part of the curve, but would see colloquial warping sensation the further out you move from that center. To test this idea I decided to take the entire trajectory I mapped out from my vector stapes, got the arc length parameterization curve, and then got the TNB values.

- When I was examining the output of the image, I realized this wouldn't work out because of the dimensional mismatch between the geometric visualization of the TNB frames and that of the tensors dimensions. TNB values are typically modeled in a 3D Cartesian coordinate system with x, y, and z axes, but the tensors were in 4D and 5D dimensions, which is how GATSegDiff and nnUNet process medical data respectively.

- The first issue was that when I went to model the frames the dimensions were being showed as parts of a 2D circle, and the TNB frames were being modeled as 2D webs rather than orthogonal vectors showing the trajectory of an arc-length curve. I figured it was just an issue with the spatial dimensions given by the code, so after multiple rounds of trouble shooting to get the 3D dimension to properly capture the arc length parameterization function, I looked at the output again.

- Another issue was emerging. While the first 3 dimensions were being properly captured, the 4th and 5th dimensions were being modeled in very strange ways. They were being modeled by artificial color scaling factors and model trajectories. This was concerning because the true dimensionality beyond the 3rd dimension was still being completely flattened. No matter which change I was making under this framework and no matter what I was emphasizing, this issue remained constant. I decided this was not going to work, so I decided to scrap the visualization and try something else.

- Instead of using vector functions and vector derivatives to model the tensor changes, I decided to examine the difference between each vector value throughout the tensor transformation sequence. I used it to try and create a multivariable function, where each variable corresponds to a different dimension within the tensor.

- Many of the same problems were occurring. The dimensions were being modeled artificially as portions of a 2D Cartesian plane. The 5D matrix wasn't being properly rendered, compromising the integrity of the visualization. So I decided that this wasn't going to work and that I have to think of something else.

- I thought back to the linear algebra videos I watched where 3Blue1Brown was modeling what it meant for vectors to span a space using arrow visualizations. I then thought back to Professor Strang's videos about how matrix A(x)=b could be used to model what vectors fit a particular subspace. I then thought about Professor Leonard's lectures on partial derivatives and that each derivative modeled the specific length which sat within a specific length of a surface. I decided this was the best way to handle this situation because the surface that would be carved by using directional derivative wouldn't be artificially scaffolded but would accurately model the reality of how the matrix twisted in space.

- So originally I had thought it would be best to graph these matrices in order to observe exactly where within the matrix changes was there a collapse occurring. The first thing I tried was to graph the matrix as continuous manifolds. I was going to do this by graphing the matrix using continuous manifolds, but then it occurred to me that along the z axis the natural mapping of the matrix was being stripped and replaced for an artificial square rooting of the two Jacobian matrices. This was problematic be it was no longer showing the natural rate of change in the tensor transformations and was now optimizizing to fill in a space the matrix may not actually accompany in real life.

- To account for this, I decided it would be better to extend my matrix row-wise to model the continuous transformation flow so that there isn't any secret artificial scaffolding going on. I'm going to take the ratios of these Det=0 and I will find those ratios because I think what's happening with GATSegDiff is that the graph, by the time is was processed and trained it had already erased so much data, and the simple UNet also erases a heavy amount of data while not even accommodating for the graphical structure, so nearly all the data has been wiped away by that moment and the model is essentially anchored in nothing. nnUNet, likewise, also suffers from data wipeouts in its training process and it doesn't come with a graphical structure like GATSegDiff does, hence why it doesn't necessarily perform better even within its own native environment.

- I finally figured out how I can actually use Womersley's number for part 2, which is to address the limitations of Murray's law, the main one being the perfect branching exponent of 3 for the parent and daughter vessels when in reality researchers garnered a range of branching exponents from 2.8 to 3.5. For Womersley's number there are no set ideals like there is for Murray's Law and deviation, as it's a dimensionless ratio dependent on dynamically scaling of the pressure gradient within that specific vessel geometric region. This means that there is an expected range of Womersley coordinates per region. Above that range but below the point of structural collapse due to that specific gradient, one can suspect there is angiogenesis occurring. Any higher than that window is a hallucination, and lower that fits the expected Womersley window of Valid Anatomy. I'd do the same check where I see what is actually being classified as valid

anatomy and what it means for clinical safety. I am also using Reynold's number in this investigation because those two calculations have a direct relationship with each other in terms of localized pressure gradient mapping in fluid flow. I manipulated parts of my Reynold's number to stretch and scale for every component of the Womerseley's number. If I get a high Reynold's number, I expect to get a high Womersley number, because that means fluid pressure in that region is significant. If it's low then the opposite holds true. Depending on these two calculations, that's how the clinical classifications would pan out.

- In order to scale my TDA coordinates to fit Reynold's number, I assigned different variables of Reynold's number to the equations I derived from my code for skeletonized calculations. L (the length) equals the equation derived for total vessel length, p (the density) is that equation divided by my branch fragmentation count, and for mu (dynamic viscosity) I decided to couple my bifurcation count with my branching tortuosity count. To do this, I noticed that my bifurcations were evaluated by the angle that was being created, and so branching of this nature typically corresponds to some sort of vector component within a triangular system. This means that at a two vessel junction I can model the vector components using a cosine function for one vector and sin function for another, given that the bifurcation is represented by theta. I put this directly within the magnitude equations for two vectors being added, as that's one of the main parts to my branching tortuosity method of which I'm basing my calculations for viscosity off of.

- As there aren't many variables in play for Reynold's number, I couldn't plug in Mean tortuosity and average branch length directly. I thought about potentially trying to directly multiply the values directly into my calculation, but I felt like then it would no longer be representative of the true value of the calculation. So I decided instead that Mean tortuosity and average vessel branch length would serve as dynamic weights that I will integrate into the overall metric by forcing Reynold's calculation to scale in the trajectory governed by these two metrics. I would make my level curves out of the Reynold's number equation I get when putting my equation together, getting the normal vector equation to it. The point of which to specify this vector would be outlined by the average vessel length as the x value and the corresponding mean tortuosity as the y value. I put it in this order because I believed that the average branch length was the independent variable that would naturally affect the tortuously within the system. All of these things together would map out the tangent plane for Reynold's number, accounting for all of my TDA in Reynold's number.

- I had wanted to solve for variables of the Womersley's number equation using my new Reynold's number equation, but then I realized none of my values directly scaled into

Womersley variables. With this understanding, I used the tangent plane from my Reynold's number manipulation scheme along with the original Womersley formula to constrain my Reynold's number calculation tangent plane to the system accompanying the original Womersley's Law. To do this effectively I thought back to Professor Leonard's videos on constrained optimization using lagrange multipliers. I set the Womersley equation as the function constrained to the tangent plant plane mapped out by Reynold's number manipulation, getting the critical points, where the peach's/valley's in the system will occur, would be within a range of x and y coordinates, which makes sense because again there isn't a hard-and-fast rule with these laws. Doing this would essentially lock in the calculation within the boundaries of Reynold's number, enabling me to audit the severity of the pressure gradient in regards to these two laws. I decided that doing this rather than trying to directly input Reynold's number directly into the calculations of Womersley's Law was the proper solution because Reynold's number doesn't naturally use angular velocity and pressure density like Womersley does, and so there wasn't a way that I could naturally use my derived formulas without modeling it within constraints of optimization.

- When implementing the code for this, I came across a major problem. Looking back at my math framework, I realized that I had no constants for Womersley's number. This was by design because I had designed the framework to have a cascading effect so that Reynold's Law directly impacts Womersley's number. But this is problematic because it means that I don't have numbers that I can put in cleanly without having to make some work around how I compute the constrained value. So what I did is that for the Womersley number components I decided that over those components of my model I took improper integrals from 0 to infinite so as to not fall into the trap of barcoding variables. This would work fine because the constrained opotomization layer would force the final value to adhere to the boundaries set by Reynold's number.

- After tracking the count of all submatrices where the Det=0, I had the computer track the number and type of eigenvalues eigenvectors that occurred within the 2x2 submatrices. I had it do this for both those submatrices where the Det=0 and when the Det=/=0. I did this because the projection matrices of the eigenvalues are meant to be perfectly aligned with the original vectors within a multidimensional space so that the error calculation is 0. In terms of creating a machine learning model, these would dramatically jack up the probability statistics of these models by keeping the margin of error low. I tracked the perfectly conforming eigenvalues, repeating eigenvalues, and complex eigenvalues (eigenvalues that are complex numbers), and "messy" eigenvalues (those that don't compute to clean values or have other structural defects to them. For GATSegDiff, nnUNet, and my SimpleUNet wrapper all of them had 0 repeated eigenvalues or eigenvalues amounting to complex numbers, and only GATSegDiff had some eigenvalues that were slightly deformed. Well, given that these models are meant to be

segmenting tumors which naturally span multiple dimensions of space where blood vessels naturally occupy planes and sometimes even overlap with each other on a single place, you'd think that there would be more non-perfect eigenvalues, so there being a grand total of 8 between all tested models showed me that they are essentially finding ways to wipe out as much data as possible with each tensor transformation to preserve only as many perfect eigenvalues and eigenvectors as possible for probability's sake.

- I just realized that there's actually another reason why these models may optimize perfectly for lambda values that a can diagonalize, and it has to do with the fact with the nature of Powers of A and differential equations. Powers of A and differential equations enable for the separate evaluation of the trajectory of matrices as powers and exponentials, making it very easy to map out the length of the corresponding curve of a given interval. Those are the things that enable models to more easily separate different types of data points within a matrix through eigenvalue diagonalization and exponential sequencing to compartmentalize and treat different data structures with care for their own coordinates and data structures, so that's probably why all of the eigenvalues and eigenvectors were all perfect with no defects whatsoever, because otherwise the easy separation mechanism would no longer hold true.